Tic-Tac-Toe Lab

Problems found and how I fixed them

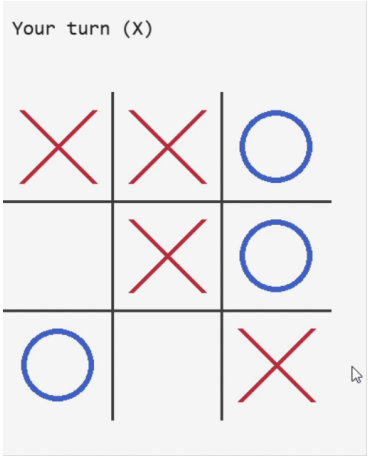
Student	Priyanka | SRN: PES2UG24CS377 | Section: G
Project	09_tic_tac_toe - Pygame Tic-Tac-Toe (player is X, computer is O)
My branch	https://github.com/Priyanka0205-CSE/09_tic_tac_toe/tree/fix-tic-tac-toe
AI chat used	https://claude.ai/share/9eaee8fe-16b5-4b57-aa8c-abe35cbfe4b1
Files changed	game/rules.py, game/game_engine.py, game/renderer.py

The starter game had bugs in its win/draw detection and move handling, and three features were missing. I first recorded the broken behavior, then fixed everything on my own branch of my fork, so the original repo is unchanged. I used Claude as my debugging partner, checked each change, and tested it by playing the game.

Task 1: Win and draw detection

1 Diagonal wins were not detected

WHAT I SAW	Three in a row on a diagonal, and no winner was announced.
WHY IT HAPPENED	**check_winner** in rules.py only checked the 3 rows and 3 columns. The two diagonals were never added.
HOW I FIXED IT	Added both diagonals to the list of lines.
BEFORE	`# NOTE: diagonals are not included here`
AFTER	`lines.append([board[0][0], board[1][1], board[2][2]])` `lines.append([board[0][2], board[1][1], board[2][0]])`



Before: X has a diagonal (top-left to bottom-right), but the game still says "Your turn".

2 A win on the last cell was shown as "Draw!"

WHAT I SAW	If the winning move also filled the ninth cell, the game said Draw instead of the winner.
WHY IT HAPPENED	**check_round_end** checked "is the board full?" before checking for a winner.
HOW I FIXED IT	Check for a winner first. It is only a draw if nobody won.

BEFORE	`if is_board_full(self.board):   # checked first`
	`    self.round_over = True`
	`    self.winner = None`
	`    return`
	`winner = check_winner(self.board)`

AFTER	`winner = check_winner(self.board)   # winner first`
	`if winner:`
	`    self.round_over = True`
	`    self.winner = winner`
	`elif is_board_full(self.board):`
	`    self.round_over = True`
	`    self.winner = None`

3 Clicks still worked after the round ended

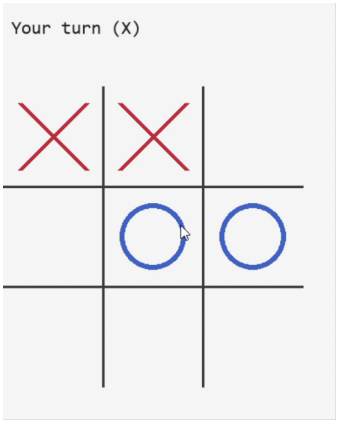
WHAT I SAW	After a win or draw, clicking empty cells kept adding new symbols.
WHY IT HAPPENED	**handle_click** never checked whether the round was already over.
HOW I FIXED IT	Return early at the start of handle_click.
AFTER	`if self.round_over:` `    return`

Task 3: Move validation

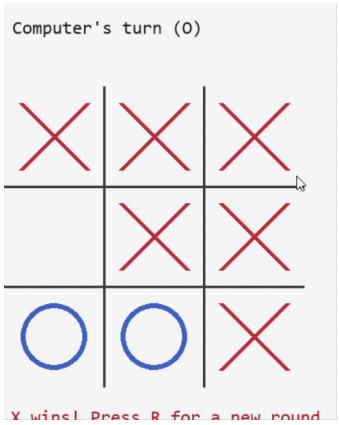
4 A filled cell could be overwritten

WHAT I SAW	Clicking a cell that already had a symbol replaced it with my own symbol.
WHY IT HAPPENED	The symbol was placed without checking whether the cell was empty (empty cells hold None).
HOW I FIXED IT	Reject the click if the cell is not empty, so the board and the turn stay exactly as they were.
BEFORE	`self.board[row][col] = self.current_player   # BUG: no check`

AFTER	`if self.board[row][col] is not None:`
	`    return`
	`self.board[row][col] = self.current_player`



Before the click: O is in the top-right cell.



After the click: the O was overwritten by an X.

Task 2: Persistent scoreboard

5 No scoreboard, and R wiped everything

WHAT I SAW	Nothing tracked wins or draws. Pressing R restarted the whole game object.
WHY IT HAPPENED	R called **__init__()**, which resets all state, and there was no place to keep the scores.
HOW I FIXED IT	Added a **scores** dictionary and a **start_new_round()** method that clears only the board and turn. check_round_end adds one point when a round ends, and the scores are drawn on screen.

AFTER	`self.scores = {'X': 0, 'O': 0, 'draw': 0}`
	`...`
	`self.scores[winner] += 1     # on a win`
	`self.scores['draw'] += 1     # on a draw`

Task 4: First-player choice and separate restart controls

6	No first-player choice, and only one reset
WHAT I SAW	The player always started, and there was one reset that could not keep the score.
WHY IT HAPPENED	The engine always started with X and had a single combined reset.
HOW I FIXED IT	Added a first-player choice (X or O). When O starts, the computer takes the first move of the round automatically. Added two separate controls: **Restart Round** (new board, scoreboard kept) and **Reset Match** (new board and scoreboard cleared).

Summary

Issue	File	Result
Diagonal wins missed	rules.py	Fixed
Win on last cell shown as draw	game_engine.py	Fixed
Clicks accepted after round end	game_engine.py	Fixed
Filled cell could be overwritten	game_engine.py	Fixed
Scoreboard	game_engine.py	Added
First-player choice	game_engine.py	Added
Restart Round / Reset Match	game_engine.py	Added

How I tested: played the game to check diagonal wins, clicks on filled cells, clicks after the round ended, the scoreboard staying after Restart Round, the first-player choice, and the scoreboard clearing on Reset Match.